

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	G.RAJU	Reg. No.:	192411065
Section / Batch:	B. TECH (CSE)	Date:	13-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. 0/1 Knapsack Optimization

Problem Understanding:

Each item can be selected **at most once**. We need to maximize profit without exceeding capacity W .

Algorithm:

Use Dynamic Programming.

$dp[i][w]$ stores the maximum profit using the first i items with capacity w .

Python Code:

```
def knapsack(values, weights, W):
    n = len(values)
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    dp[i - 1][w],
                    values[i - 1] + dp[i - 1][w - weights[i - 1]]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][W]
```

```
n = 3
```

```
values = [60, 100, 120]
```

```
weights = [10, 20, 30]
```

```
W = 50
```

```
print("Maximum Profit =", knapsack(values, weights, W))
```

Output:

Maximum Profit = 220

Complexity:

- Time: $O(n \times W)$
- Space: $O(n \times W)$

Test Cases: Handles zero capacity, items heavier than capacity, and normal cases.

Q2. Unbounded Knapsack

Problem Understanding:

Unlike 0/1 Knapsack, each item can be selected **multiple times**. The objective is to maximize profit without exceeding capacity.

Algorithm:

Use a one-dimensional Dynamic Programming array where $dp[w]$ represents the maximum profit for capacity w .

Python Code:

```
def unbounded_knapsack(values, weights, W):
```

```
    n = len(values)
```

```
    dp = [0] * (W + 1)
```

```
    for w in range(W + 1):
```

```
        for i in range(n):
```

```
            if weights[i] <= w:
```

```
                dp[w] = max(
```

```
                    dp[w],
```

```
                    values[i] + dp[w - weights[i]]
```

```
                )
```

```
    return dp[W]
```

```
n = 3
```

```
values = [10, 30, 20]
```

```
weights = [5, 10, 15]
```

```
W = 100
```

```
print("Maximum Profit =", unbounded_knapsack(values, weights, W))
```

Output:

Maximum Profit = 300

Complexity:

- Time: $O(n \times W)$
- Space: $O(W)$

Test Cases: Handles capacity 0, repeated item selection, and items heavier than capacity.

Q3. Traveling Salesman Problem (TSP)

Problem Understanding:

The salesman must visit every city exactly once and return to the starting city while minimizing total travel cost.

Algorithm:

Use the **Held-Karp Dynamic Programming** approach with bitmasking.

$dp[mask][i]$ represents the minimum cost to visit the cities in mask and finish at city i .

Python Code:

```
def tsp(cost):
    n = len(cost)
    INF = float('inf')

    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0

    for mask in range(1 << n):
        for u in range(n):
            if not (mask & (1 << u)):
                continue

            for v in range(n):
                if mask & (1 << v):
                    continue

                new_mask = mask | (1 << v)
                dp[new_mask][v] = min(
                    dp[new_mask][v],
                    dp[mask][u] + cost[u][v]
                )

    full_mask = (1 << n) - 1

    return min(
        dp[full_mask][i] + cost[i][0]
        for i in range(1, n)
    )

cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

print("Minimum Cost =", tsp(cost))
```

Output:

Minimum Cost = 80

Complexity:

- Time: $O(n^2 \times 2^n)$
- Space: $O(n \times 2^n)$

Test Cases: Handles different city counts, symmetric costs, and complete travel matrices.

Q4. Floyd-Warshall Algorithm**Problem Understanding:**

The algorithm finds the **shortest paths between every pair of vertices** in a weighted graph.

Algorithm:

For every intermediate vertex k , check whether travelling through k gives a shorter path:

$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

Python Code:

```
def floyd_warshall(graph):
    n = len(graph)
    INF = float('inf')

    dist = [
        [INF if x == "INF" else x for x in row]
        for row in graph
    ]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] != INF and dist[k][j] != INF:
                    dist[i][j] = min(
                        dist[i][j],
                        dist[i][k] + dist[k][j]
                    )

    return dist

graph = [
    [0, 5, "INF"],
    [2, 0, 3],
    ["INF", "INF", 0]
```

```
]
```

```
result = floyd_warshall(graph)
```

```
for row in result:
```

```
    print(*[
        "INF" if x == float('inf') else x
        for x in row
    ])
```

Output:

```
0 5 8
```

```
2 0 3
```

```
INF INF 0
```

Complexity:

- Time: $O(n^3)$
- Space: $O(n^2)$

Test Cases: Handles disconnected vertices, direct paths, and paths improved through intermediate vertices.

Q5. Optimal Binary Search Tree (OBST)

Problem Understanding:

Keys have different search frequencies. The goal is to construct a BST with **minimum expected search cost**, placing frequently searched keys closer to the root.

Algorithm:

Use Dynamic Programming.

For every interval of keys, try each key as the root and select the minimum-cost arrangement.

Python Code:

```
def optimal_bst(keys, freq):
    n = len(keys)

    dp = [[0] * n for _ in range(n)]

    prefix = [0] * (n + 1)
    for i in range(n):
        prefix[i + 1] = prefix[i] + freq[i]

    def range_sum(i, j):
        return prefix[j + 1] - prefix[i]

    for i in range(n):
```

```
dp[i][i] = freq[i]
```

```
for length in range(2, n + 1):
```

```
    for i in range(n - length + 1):
```

```
        j = i + length - 1
```

```
        dp[i][j] = float('inf')
```

```
        total_freq = range_sum(i, j)
```

```
        for r in range(i, j + 1):
```

```
            left = dp[i][r - 1] if r > i else 0
```

```
            right = dp[r + 1][j] if r < j else 0
```

```
            cost = left + right + total_freq
```

```
            dp[i][j] = min(dp[i][j], cost)
```

```
return dp[0][n - 1]
```

```
keys = [10, 12, 20]
```

```
freq = [34, 8, 50]
```

```
print("Minimum Cost =", optimal_bst(keys, freq))
```

Output:

Minimum Cost = 142

Complexity:

- Time: $O(n^3)$
- Space: $O(n^2)$

Test Cases: Handles one key, multiple keys with different frequencies, and equal frequencies.